Unidad Temática N°2

~ Principios S.O.L.I.D. ~

"Cualquiera puede escribir código que una computadora pueda entender. Los buenos programadores escriben código que los humanos pueden entender.".

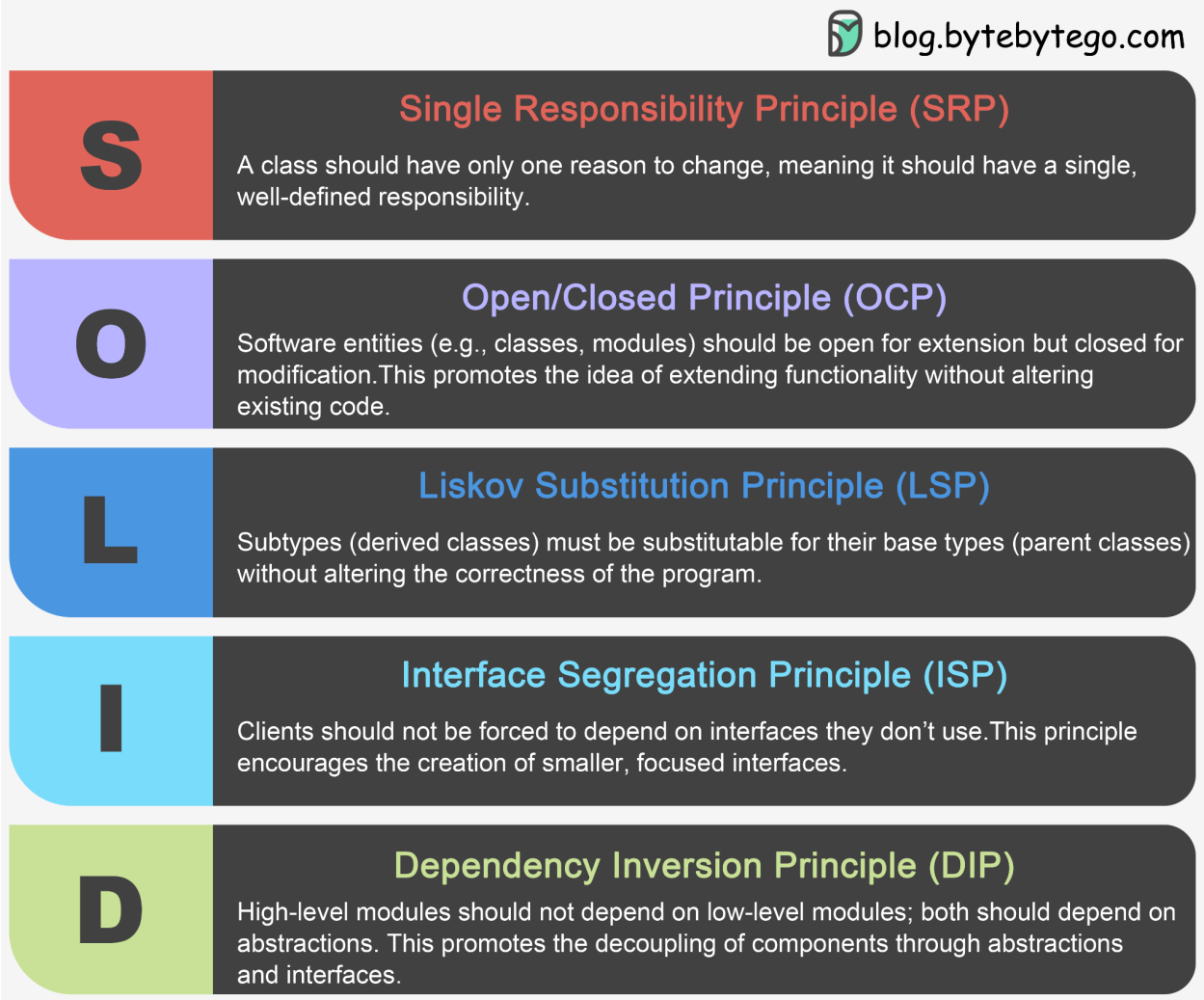
~ Martin Fowler

Principios S.O.L.I.D.

Introducción

Los principios S.O.L.I.D, son 5 postulados que fueron mencionados por primera vez al principio de los 2000 por Robert C. Martin en la presentación del paper Design Principles and Design Patterns.

Luego estos conceptos fueron desarrollados por Michael Feathers, dándole el nombre de principios SOLID.



Importancia de los principios SOLID en el desarrollo de software

Los principios de diseño, como los principios SOLID, desempeñan un papel fundamental en el proceso de desarrollo de software por varias razones:

Mantenibilidad: seguir unos principios de diseño sólidos hace que el código sea más fácil de mantener. Cuando el código está bien estructurado y se adhiere a estos principios, resulta más fácil identificar y solucionar problemas, añadir nuevas funciones y realizar mejoras sin causar consecuencias no deseadas.

Escalabilidad: Un software bien diseñado es escalable. Puede adaptarse a los cambios y al crecimiento de los requisitos sin necesidad de grandes modificaciones ni volverse cada vez más complejo.

Reusabilidad: La adhesión a los principios de diseño a menudo conduce a un código que es más reutilizable. Los componentes reutilizables ahorran tiempo y esfuerzo en el desarrollo y las pruebas.

Colaboración: Los principios de diseño proporcionan un marco común de trabajo para los desarrolladores. Este entendimiento común fomenta la colaboración y reduce los malentendidos entre los miembros del equipo.

Reducción de errores: Seguir los principios de diseño ayuda a identificar y mitigar los errores comunes de programación y los fallos de diseño. El resultado es un menor número de errores y un software más robusto.

Principios S.O.L.I.D

Single Responsibility (Responsabilidad única)

Este principio establece que una clase sólo debe tener una responsabilidad, esto nos lleva a pensar que por tener una única responsabilidad sólo debe tener una única razón para cambiar, para ser modificada.

Beneficios obtenidos al aplicar esta principio:

Pruebas: una clase con una única responsabilidad tendrá (probablemente) menos casos de prueba.

Reducción del acoplamiento: menos funcionalidad en una sola clase tendrá menos dependencias hacia otras clases.

Ejemplo

```
using System;

public class Book
{
    private string name;
    private string author;
    private string text;

    // Constructor
    public Book(string name, string author, string text)
    {
        this.name = name;
        this.author = author;
        this.text = text;
    }

    // Getters y Setters
    public string Name
    {
        get => name;
        set => name = value;
    }

    public string Author
    {
        get => author;
        set => author = value;
    }

    public string Text
    {
        get => text;
        set => text = value;
    }

    // Reemplazar palabra en el texto
    public string ReplaceWordInText(string word)
    {
        // En C#, Replace reemplaza todas las ocurrencias
        return text.Replace(word, text);
    }

    // Verificar si una palabra está en el texto
```

```
    public bool IsWordInText(string word)
    {
        return text.Contains(word);
    }

    // Imprimir el texto en consola
    public void PrintTextToConsole()
    {
        Console.WriteLine(text);
    }
}
```

Ejemplo 42.1

¿Cuál es el problema en la clase Book?

Que no sólo manipula el contenido del libro sino que además permite formatear e imprimir su contenido.

¿Qué ocurre si se requiere cambiar el formato? ¿Qué pasa si se requiere imprimir a un archivo?, etc.

¿Cómo se soluciona?

```
using System;

public class Book
{
    private string name;
    private string author;
    private string text;

    // Constructor
    public Book(string name, string author, string text)
    {
        this.name = name;
        this.author = author;
        this.text = text;
    }

    // Propiedades
    public string Name
    {
        get => name;
        set => name = value;
    }
```

```
    public string Author
    {
        get => author;
        set => author = value;
    }

    public string Text
    {
        get => text;
        set => text = value;
    }

    // Reemplazar palabra en el texto
    public string ReplaceWordInText(string word)
    {
        // Nota: aquí se reemplaza la palabra por todo el texto (igual que en TS).
        // Lo más útil sería reemplazar por otra palabra, pero mantengo la lógica
original.
        return text.Replace(word, text);
    }

    // Verificar si una palabra está en el texto
    public bool IsWordInText(string word)
    {
        return text.Contains(word);
    }
}

public class BookPrinter
{
    // Imprimir texto en consola
    public void PrintTextToConsole(string text)
    {
        Console.WriteLine(text);
    }

    // Simulación de impresión en otro medio
    public void PrintTextToAnotherMedium(string text)
    {
        // Aquí podrías implementar la lógica para enviar el texto a otro
dispositivo
        Console.WriteLine($"[Otro medio] {text}");
    }
}
```

Ejemplo 42.2

Open for extension, Close to modification (abierto a la extensión, cerrado a la modificación)

Este principio menciona que las clases deben estar abiertas para su extensión pero cerradas para su modificación. Si hacemos esto, evitamos modificar el código existente y causar potenciales errores en una aplicación.

La excepción a la regla recae sobre la corrección de errores.

Ejemplo

Continuando con el ejemplo 42.1 y 42.2, suponga que se agrega el requerimiento de guardar en un archivo de texto el contenido del libro. Tendríamos el siguiente código:

```
using System;
using System.IO;

public class BookPersistence
{
    // Guardar el texto del libro en un archivo
    public void SaveTextToFile(Book book, string filePath)
    {
        try
        {
            File.WriteAllText(filePath, book.Text);
            Console.WriteLine($"Texto guardado en {filePath}");
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine($"Error al guardar el archivo: {ex.Message}");
        }
    }
}
```

Ejemplo 42.3

Tiempo después se solicita incorporar una base de datos y guardar el texto del libro en la misma para mejorar las búsquedas.

El primer acercamiento a la solución podría ser agregar un método en la clase *BookPersistence* que permita guardar el libro en una base de datos. Pero esto, claro está, rompería el principio Open/Close.

¿Cómo se soluciona?

El uso de interfaces (o en su defecto clases abstractas) podría ayudar a alcanzar la solución.

```
using System;
using System.IO;

public interface IBookPersistence
{
    void Save(Book book);
}

public class TextFilePersistence : IBookPersistence
{
    public void Save(Book book)
    {
        // Guardar el texto del libro en un archivo
        string filePath = $"{book.Name}.txt";
        File.WriteAllText(filePath, book.Text);
        Console.WriteLine($"Texto del libro guardado en archivo: {filePath}");
    }
}

public class DbPersistence : IBookPersistence
{
    public void Save(Book book)
    {
        // Simulación de guardar el texto en una base de datos
        Console.WriteLine($"Texto del libro '{book.Name}' guardado en la base de
datos.");
    }
}
```

Ejemplo 42.4

En el ejemplo se puede apreciar que se tuvo que refactorizar el código ya que en el diseño inicial no se tuvo presente el principio Open/Close, pero el código que guarda el texto a un archivo de texto no fue modificado.

Otra ventaja es que ahora la aplicación puede ser extendida, agregando nuevas formas de guardar el contenido de un libro sin alterar el código existente.

Por último, no hay que perder de vista la ganancia obtenida de **desarrollar orientado a interfaces**. El código automáticamente pasó a ser polimórfico, con lo que podemos intercambiar las clases *TextFilePersistence* y *DbPersistence* y todas las que se agreguen sin que el código cliente se vea afectado.

Liskov Substitution (Sustitución de Liskov)

Bárbara Liskov introdujo este principio en 1987, y para explicarlo de forma sencilla, podemos decir que este principio establece que si B es una subclase de A, A debe poder ser reemplazada por B sin que la aplicación vea afectada su comportamiento.

El objetivo final de este principio, es poner de manifiesto el uso de clases polimórficas para clases que pertenezcan a una determinada jerarquía. Esto nos daría la ventaja de poder extender la aplicación sin que el código cliente se vea afectado.

```
using System;

public abstract class Animal
{
    public abstract void MakeNoise();
}

public class Dog : Animal
{
    public override void MakeNoise()
    {
        Console.WriteLine("bow wow");
    }
}

public class Cat : Animal
{
    public override void MakeNoise()
    {
        Console.WriteLine("meow meow");
    }
}

public class DumbDog : Dog
{
    public override void MakeNoise()
    {
        throw new Exception("I can't make noise");
    }
```

```
}

class Program
{
    static void Main()
    {
        Animal dog = new Dog();
        dog.MakeNoise(); // bow wow

        Animal cat = new Cat();
        cat.MakeNoise(); // meow meow

        Animal dumbDog = new DumbDog();
        try
        {
            dumbDog.MakeNoise(); // lanza excepción
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error: {ex.Message}");
        }
    }
}
```

Ejemplo 42.5

En este ejemplo vemos que no se cumple el principio de Liskov, ya que si usaramos la una instancia de la clase `DumbDog` para reemplazar la clase `Dog`, nuestra aplicación dejaría de funcionar.

Interface Segregation (Separación de interfaces/conceptos)

Este principio simplemente significa que las interfaces más grandes (con muchos métodos) deben dividirse en otras más pequeñas. Al hacerlo, podemos asegurarnos de que las clases que las implementan sólo necesitan preocuparse por los métodos que les interesan. En otras palabras, no forzar la implementación de métodos que una clase no necesita tener.

```
using System;

public interface IBookPrinter
{
    void Print(string text);
}

public interface IPaperBookPrinter
```

```
{
    bool HasPaper();
}

public class BookPrinter : IBookPrinter
{
    public void Print(string text)
    {
        // Imprime el texto por consola
        Console.WriteLine(text);
    }
}

public class PaperBookPrinter : IBookPrinter, IPaperBookPrinter
{
    private bool paperAvailable;

    public PaperBookPrinter(bool hasPaper)
    {
        paperAvailable = hasPaper;
    }

    public bool HasPaper()
    {
        return paperAvailable;
    }

    public void Print(string text)
    {
        if (HasPaper())
        {
            Console.WriteLine($"[Impresión en papel] {text}");
        }
        else
        {
            Console.WriteLine("No hay papel disponible para imprimir.");
        }
    }
}
```

Ejemplo 42.6

Dependency Inversion (Inversión de dependencia)

El último principio sugiere que las clases deben depender de abstracciones y no de clases concretas.

Este principio enfatiza la necesidad de tener interfaces que nos ayuden a definir y comunicarnos con las clases concretas.

La ventaja de esto es que se oculta la implementación real de la clase A a la clase B. Entonces, si la clase A cambia, la clase B no necesita ser modificada.

Una vez más lo que tratamos de hacer es desacoplar el código.

Veamos un ejemplo concreto de esto.

```
public class Computer
{
    private readonly StandardKeyboard keyboard;
    private readonly Monitor monitor;

    public Computer()
    {
        monitor = new Monitor();
        keyboard = new StandardKeyboard();
    }
}
```

Ejemplo 42.7

Si bien este ejemplo funciona, podemos observar 2 problemas:

1. Estamos acoplando las clases Monitor, StandardKeyboard y Computer
2. Si necesitamos "actualizar" nuestro monitor o nuestro teclado, no podremos.

¿Cómo se soluciona?

```
interface Keyboard {}

interface Monitor {}

class Computer {
  private keyboard: Keyboard;
  private monitor: Monitor;

  constructor(keyboard: Keyboard, monitor: Monitor) {
       this.keyboard = keyboard;
       this.monitor = monitor;
  }
}
```

```
public class StandardKeyboard : IKeyboard
{
    public void Type(string text)
    {
        Console.WriteLine($"Typing: {text}");
    }
}

public class Monitor : IMonitor
{
    public void Display(string content)
    {
        Console.WriteLine($"Displaying: {content}");
    }
}
```

Ejemplo 42.8

Ahora las clases están desacopladas. Si queremos, podemos cambiar fácilmente el tipo de teclado en nuestra máquina con una implementación diferente de la interfaz. Podemos seguir el mismo principio para la clase Monitor.

Bibliografía

- Luis Joyanes Aguilar, Ignacio Zahonero Martínez. Programación en C. Segunda Edición. España: McGRAW-HILL/INTERAMERICANA DE ESPAÑA. S.A.U., 2005. ISBN-84: 481-9844-1.
- Brian W. Kernighan, Rob Pike. La práctica de la programación. Pearson Educación. Méjico (2000)
- Debugging with CodeBlocks: http://wiki.codeblocks.org/index.php?title=Debugging_with_Code::Blocks
- Bibliotecas.
    - https://es.wikipedia.org/wiki/Biblioteca_(inform%C3%A1tica)
- Enlace dinámico
    - https://es.wikipedia.org/wiki/Enlace_din%C3%A1mico
- Enlace estático
    - https://es.wikipedia.org/wiki/Enlace_est%C3%A1tico